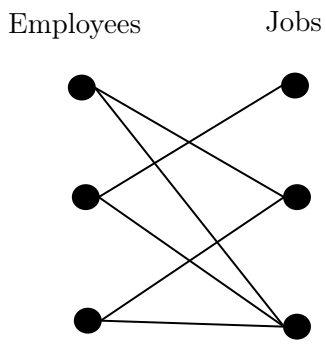
Matching

A small blog post by Yves Jäckle, made in June 2023

1 Maximum matching

Bipartite matching
Suppose you're managing a team that's developing a video game. You have a set of employees and a set of intermediate jobs to be performed by them to make the game. In your team, employees have different skills and different levels of experience, so that some might not be able to perform certain jobs (or at least not well or not in a reasonable time span). For example, you could have two graphics designers that know two of the following skills: character design, landscape design and animation.
How could you get a better picture of the situation ?

On way to represent the situation is the following: we represent employees and jobs by the vertices of a graph, grouping employee vertices and job vertices together, and we add an edge between an employee and a job if the job can be performed by that employee.



This graph will be bipartite:

Bipartite graph:

A graph is bipartite if its vertices are partitioned into A and B and for all edges $\{u, v\}$ of the graph, $u \in A$ and $v \in B$ or vice versa.

We'll assume that one employee can work on one job only and that a job can be worked on by one employee only.
Your goal is to get all the jobs done, if doing so is possible, by a clever assignment of employees to jobs.
So when we assign employee u to job v, we can represent this on our graph by highlighting edge $\{u, v\}$. By our assumptions, each vertex can be incident to at most one highlighted edge. We are therefore looking for a:

Matching:

A matching M is a subset of edges E of a graph such that no two edges of M have a vertex in common.

To be precise, we are looking for a **maximum matching**, which is a matching of greatest size among all possible matchings. If all employees and jobs are matched in such a maximum matching, then we'll have an assignment for which all jobs can be completed. Otherwise, there is no way of completing all jobs with our team. So how do we find a maximum matching in a bipartite graph ?

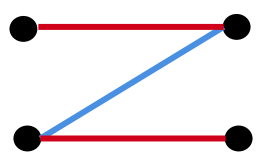
A first idea could be to add edges with different endpoints until all edges have been considered.

Maximal matching
Input: a graph (V, E)
Start with $M := \emptyset$
Take note of the matched vertices $C := \emptyset$
For $\{u, v\} \in E$:
    If $u \notin C$ and $v \notin C$:
        Add $\{u, v\}$ to M
        Add u and v to C
Output: a maximal matching M

There are no spelling mistakes in this description: we output a **maximal matching**, which is a matching to which no edges from E can be added to produce another matching.

Before getting into the subtle difference between maximum and maximal, we'll check the algorithm.
The loop invariant is that M stays a matching: at each iteration, we add a edge only if its endpoints aren't matched already. After termination, when all edges have been considered, no more edge can be added to M and so that it remains a matching. To see this, recall the iteration in which the edge to be added was considered: we discarded it, as one of its endpoints was already matched. So it's endpoints are still matched in M and we can't add it without breaking the matching property.

Are maximal matchings also maximum ?
If they were, our previous algorithm would solve our problem. Unfortunately, they aren't. The problem lies in the fact that an unfortunate ordering of the edges in our algorithm can led to sub-optimal choices, as the following example shows:



In the case of this Z-shaped graph, the matching represented by the blue edge is maximal: adding any of the two remaining edges is impossible, as in both cases, one of their endpoints is already matched. The red edges represent a matching of greater size, so the blue matching is maximal, but not maximum.

Ex.MaMu: There is however an interesting relation between the sizes of maximal and maximum matching: if M_a is maximal and M_u is maximum, then $|M_u| \leq 2|M_a|$. Prove this (later).
So the maximal matching algorithm provides us with an upper bound on the size of a maximum matching.

If we analyse the previous counterexample, we may get an idea as to how to improve on a given matching. This way, we don't have to start solving the problem from scratch again.

If we had a way to detect that the blue edge's endpoints are connected to two different vertices that aren't matched by the blue matching, we could then perform a switch operation that discards the blue

edge and adds the red ones. Let's attempt to generalise this.
To increase the size of the matching, we need to increase the number of matched vertices. So we can start by looking for vertices that haven't been matched by our original matching. If we find one, we look at its incident edges and disjoin two cases: either the other endpoint of that edge isn't matched, in which case we can add that edge to increase the matching, or it isn't, in which case there is an edge from the matching incident to the other endpoint.
The previous counterexample shows that the second case isn't hopeless: we follow the edge from the matching and hope that the next endpoint on our path is connected to an unmatched vertex.
We can iterate this reasoning and hope to end up with the following structure:

Augmenting paths:

An augmenting path P for a matching M is a path who's start- and endpoint aren't matched by M and who's edges alternate between being in $E \backslash M$ and in M.
So in particular, augmenting paths start and end with edges from $E \backslash M$ and have odd length.

For example, the previous counterexample represented an augmenting path of length 3.

Given an augmenting path, we can improve the matching:

Augmenting matchings:

Given an augmenting path P for a matching M, the symmetric difference $M \Delta P$ (the edges that aren't common to both) is a matching of greater size then M.

Proof: To verify that we end up with a matching, we look at the vertices and disjoin cases. If the vertex isn't on P or is one of its endpoints, it will be incident to the at most one edge of $M \Delta P$ since M is a matching and the endpoints of P aren't matched by M. If it is in the interior of the path P, then it's incident to and edge that is common to M and P, which will be deleted, and also to an edge that is in P only, which will be added, but to no other edges, as M is a matching and P a path. So these points will still be matched in $M \Delta P$.
By the nature of augmenting paths, there is one more edge of $E \backslash M$ on them then from those of M.
So $|M \Delta P| = |M| + 1 > |M|$ and we've obtained a bigger matching. $\square$

So we can look for augmenting paths in a given matching to improve on it. We can then repeat this step, knowing that at some point, there can't be any augmenting paths anymore, since the size of the matching keeps increasing, but is bounded by $|E|$.
Before clarifying how we can find augmenting paths (if there are any) in practice, we should address the elephant in the room. If we can't find any further augmenting paths, is the matching maximum ?

Characterising maximum matchings through augmenting paths:

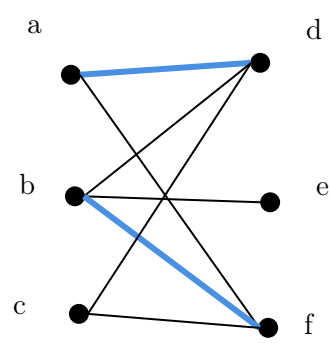
M is a maximum matching $\Leftrightarrow$ there are no augmenting paths for M.

Proof: The contrapositive states that M isn't a maximum matching $\Leftrightarrow$ there is an augmenting path for M, and it's what we'll prove. We've already taken care of $\Leftarrow$ in the previous result.
Proving $\Rightarrow$ requires more care.

We'll investigate the difference between matchings M and M' with $|M| < |M'|$ in hope of finding some way to augment M. In fact we'll study the difference quite literally: when looking at the edges of $M \Delta M'$ only, the vertices of the graph come in 3 categories: either they aren't incident to any edges, or they are incident to 1 or 2 edges, where in the last case, the two edges have to come from different matchings.
So if we walk along the edges of $M \Delta M'$, we will trace out nothing but disjoint cycles or paths, depending of whether the walk ends where it started or ends in a vertex of degree 1. All other walks are impossible, as one would have to encounter a vertex of degree at least 3.
The cycles have to be even, since they can only alternate between edges of $M \backslash M'$ or $M' \backslash M$. So on them we count as many edges of $M \backslash M'$ as those of $M' \backslash M$.
The same is true for even paths. But for odd paths, things start to get interesting. These paths are augmenting paths for the matching which leaves the paths endpoints unmatched. So if we find an augmenting path for M we have the result we looked for. Does there have to exist such a path ?
If there wasn't, either all paths would be even, so that by our counting $|M| = |M'|$, or all odd paths would their majority of edges in M, so that $|M| > |M'|$. But $|M| < |M'|$ by assumption, so this can't happen. There must exist an augmenting path for M. $\square$

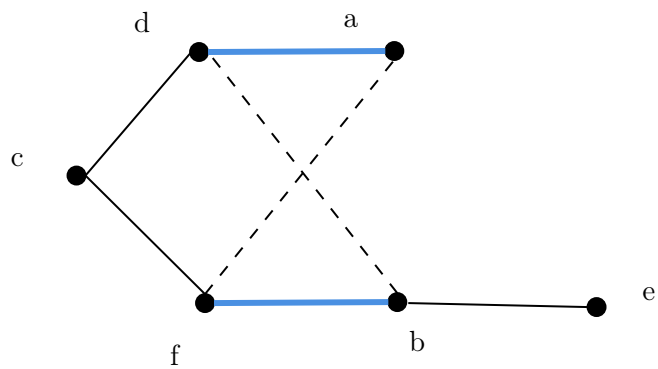
This is great news! It means that the approach of successively improving matchings with augmenting paths until there aren't any will output a maximum matching.
So all that's left for us to do is to develop a way of systematically finding augmenting paths or determining that none exist.

For example, we consider the following bipartite graph and matching (blue edges):



Augmenting paths start and end at vertices that aren't matched by the matching. So the first step will be to keep track of the un-/matched vertices. In the example above, we could start at c, which is unmatched. Next, we have to find a path that alternates between edges on and off the matching and ends at another unmatched vertex. We will do so with a graph search close to breadth-first search.

First, we consider the neighbors of c, d and f. They are matched, so we follow the unique edges to get to a and b respectively, ignoring the other edges. Finally, a only leads us to a vertex we've already visited, while b leads us to an unmatched vertex, e.



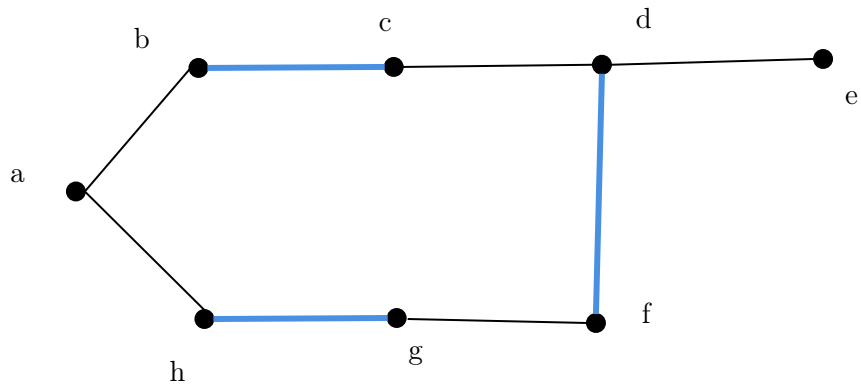
So we find the augmenting path (c, f, b, e). How do we proceed in general ?

Here's an algorithm that looks for augmenting paths starting at v.

Augmenting Paths
Input: a graph (V, E) and a matching M (and a set C of vertices matched by M)
1. Check if $V \backslash C$ has size at least 2 and take an unmatched vertex v from it.
2. Construct a tree routed at v as follows:
3. Add the neighborhood of v
4. For the leaves u of the tree, consider two cases:
5. If the the distance to v is even, add the unmatched edges of $\{u, w\}$ that lead to vertices not already in the tree. Then check if one of the w is unmatched. If it is, the w-v-path is augmenting, so we return it. If the distance is odd (and by iterative assumption, u is matched), add the unique edge of u in the matching M to the tree.
6. Iterate step 5. until all vertices reachable from v have been considered (that is, if there are no more unmatched edges leading to new vertices in step 5.)

We construct a tree, layer by layer, alternating between edges form the matching and edges not in the matching. The immediate question is: if an augmenting path starting at v exists, does our algorithm find one ?

To see that it does, notice that the tree we construct contains all the vertices that are reachable from v and that all paths on it are alternating between edges form the matching and edges not in the matching. The fact that we are building a tree requires a subtle justification of great importance.
The structure is surely connected, as we repeatedly get neighborhoods, so that all vertices have a path to v, by induction. Can there be cycles ?
When considering the edges that aren't in the matching, we exclude those that lead to vertices that already are in the tree, preventing a cycle from being closed. But we don't check this for the edges in the matching, which can lead to problematic situations, such as this one:



Here, our construction would fail once we reach d and f. The failure is due to the fact that the edge from the matching is connecting two vertices from our tree, thereby closing a cycle.
Even more embarrassingly, there is an augmenting path we fail to get: (a, h, g, f, e).
Can we fix this ? Or does this problem ever appear in our context ?

Will investigate how to fix this problem for general matchings in the next chapter. Now however, we're still in the setting of a bipartite graph. By investigating carefully, we notice that the cycles closed by edges of the matching must be odd, as the paths to the endpoints of the closing edge are augmenting, thus odd. But bipartite graphs can't contain odd cycles: if there was one, it would have to contain an odd number of vertices, which belong to one of two partition sets. So two neighbors on the cycle would have to be in the same partition set, yet this is impossible by bipartiteness.

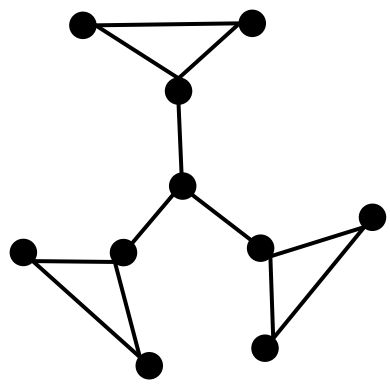
Ex.BOC: It turns out that the property of not having odd cycles implies that the graph is bipartite. Show how to construct a bipartition from this property.

So in the case of a bipartite graph, we don't close cycles when adding a layer to our tree corresponding to edges from the matching. We therefore conclude that we really are building a tree.

We now can settle the main question of whether we find an augmenting path starting at v, if one exists. An augmenting path P starting at v has its first edge in the tree, since the tree contains the neighborhood of v. We then show that all further vertices on P can be reached on our tree inductively. The induction step disjoins two cases. For the current vertex on P, if the previous edge on P is from the matching, then there is a unique predecessor, the vertex it's matched with, which is both on the tree and on the path. If the previous edge isn't from the matching, it may also be in the tree but if it isn't, it's only because there is another edge from the tree that connects to the endpoint. To be precise, the edge would have been added in step 5 of our algorithm, since it's a neighbour to a matched vertex, only if it didn't close a cycle: this implies that the vertex was already in the tree.

In conclusion, we can there fore use the following algorithm to find maximum matchings in bipartite graphs, as is guaranteed by the characterisation of maximum matchings by augmenting paths.
As a final remark, note that it is possible that two unmatched vertices exist, but no augmenting path does. This may come a bit as counter-intuitive, but it is possible:

Bipartite Matching
Input: a graph (V, E)
1. Start with a matching, for example the output of **Maximal Matching**, or $\emptyset$,
2. Use **Augmenting Paths** starting from an unmatched vertex v
3. Augment the matching by the augmenting path (take the symmetric difference)
4. Reiterate 2. and 3. until no more augmenting paths can be found for all exposed vertices.
Output: a maximum matching M



In a matching of this graph, we may take only one edge per triangle, leaving one vertex unmatched. The only possibility for that vertex to be matched is when we match it to the center of the "wheel", which we may do once at most. So there will always be at least two unmatched vertices in a matching, in particular in a maximum one, which doesn't have augmenting paths.

Ex.EV: When looking for augmenting paths, we've looked for one starting at all exposed vertices in each iteration. But is this necessary ? If no augmenting path starting at an exposed u was found in one iteration, could it be the start of an augmenting path in the next ?

We conclude this section with some nice exercises on bipartite matching:

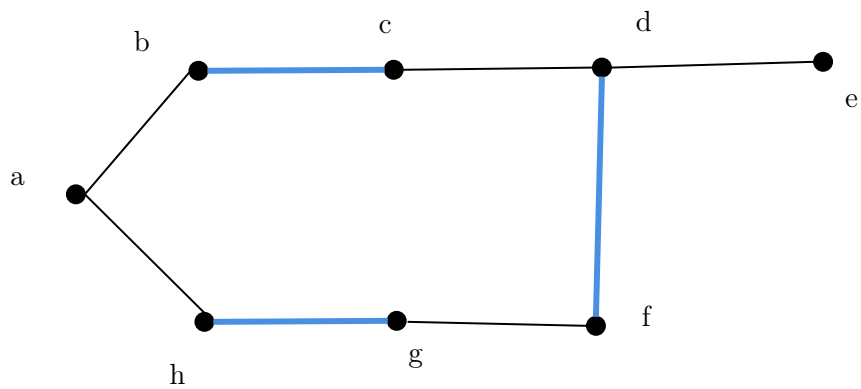
Ex.MD: We're given some planar shape made of identical square tiles: we can model this by a finite subset of $\mathbb{Z}^2$ by identifying squares with their lower left corner. We're asked to cover this shape with dominoes such that as many tiles are covered as possible. How do we proceed ?

General matching
We now investigate how to fix the problem we could afford to ignore in the previous section, because the graph was bipartite. This will allow us to find maximum matchings in non-bipartite graphs.

Matching in general graphs is of interest when the objects to be matched aren't in two categories (employees and jobs in our previous example). For example, consider a school class that is playing badminton in 1 vs. 1 matches. Student pairs inform the sports teacher that they wish to face each other in a match: based on these preference, how can the sports teacher get as many desired matches playing simultaneously ?
If we represent the students by vertices connected by an edge if the corresponding pair wishes to face off in a match, we are looking for a maximum matching in a general graph, which will correspond to the student pairs we can have playing simultaneously.

Recall the problem in our approach to find augmenting paths. We ran into trouble when an edge of the matching connected two vertices already in the tree. This edge of the matching closed a cycle in

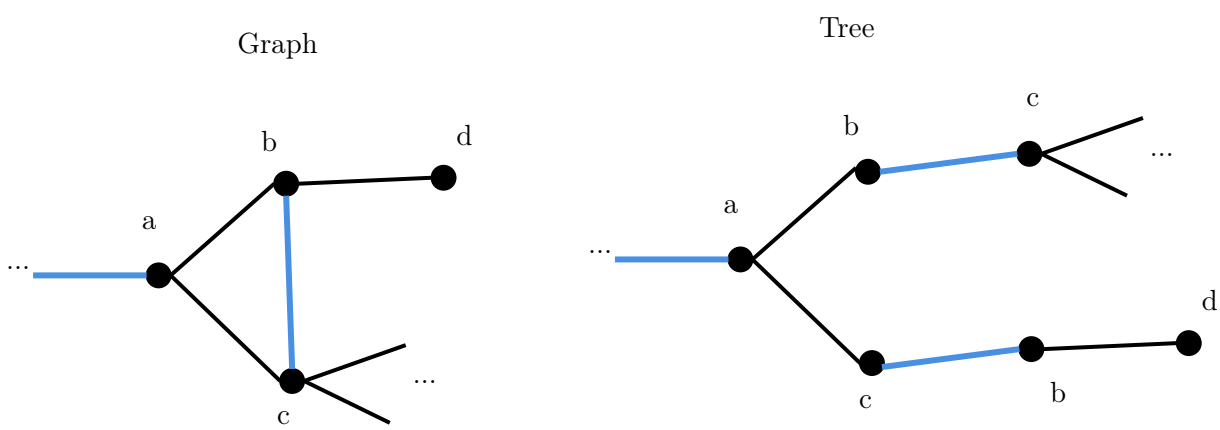
our tree, which is called a **blossom**. Blossoms have a particular structure: they are odd cycles that are alternating between edges from the matching and edges not in the matching, except at a single point, called the base of the blossom. The name comes from a particular way to deal with them, which we won't follow here: blossoms are shrunk at first, and then we un-shrink them (they "bloom") once we've found an augmenting path, and handle this un-shrinking so as to maintain the path.
In such a case, an augmenting path could still exist, as was shown in the example:



What do we do in such a situation ?
What can be said about the augmenting paths passing through such cycles ?

The augmenting paths passing through blossoms in the graph will "jump branchesïn the tree we use to search for augmenting paths. For example, in the above figure, the alternating path of the lower branch, from a to f will jump to the higher one, that from a to d to continue.

One way to fix this is to "doublethe edge in the tree, as we represent here:



This helps getting a clear picture. It also maintains the whole point of the algorithm: if the graph has an augmenting path, we can find it in the tree we construct, rooted at the start of the augmenting path.
The argument given to see this is the same as in the previous section, since the matched edge will always be added to the tree, possibly twice if it connected two vertices already in the tree.

We can fix our algorithm "**Augmenting Paths**"with one line of pseudo-code:

5. If the endpoints of the matched edge are both in the tree, add it twice to the tree with the order of the endpoints reversed.

Ex.BlBl: Though our "doublingöf the edge in the tree fits in some way the image of the blossom blooming, we'll ask you to prove a property that has given the name to blossoms. To do so, we define G/B for a blossom B wrt. M to be the graph with the vertices of B repaced by a single vertex b, such that all edges inicident to vertices of B are now incident to b. Show that M is still a matching in G/B and that if G/B has an augmenting path wrt. M, the so does G.

Minimum edge cover:
Consider the problem of finding the minimum size set of edges of a graph, so that all vertices are incident to at least one edge of the set. A first observation to make about such a set is that it can't contain any path of length greater the 3: if it did, we could delete a middle edge of the path and retain the edge cover property, as the endpoints of the delete edge are still covered by the neighbouring edges of the path; this would contradict minimality. This means that a minimum edge cover is a node-disjoint union of stars.
If we remark further that by selecting one edge per star we obtain a matching who's size is the number of stars, we can develop the following intuition. Having stars with many edges seems inefficient, as all these edges are used to cover the same central node. Also, having many stars seems to decrease the number of edges per star. So if we had a way of turning matchings into edge covers, we could look for a maximum matching, and hope that the resulting edge cover has few egdes.
We can turn a matching into an edge cover, by doing the following for each uncovered vertex: we add an edge linking it to one of the matched vertices. To see that doing this is always possible, we must actually assume that the graph has no isolated vertices and that the matching is maximum. Then, each vertex has an edge incident to it, which leads it to a matched vertex, as otherwise, this edge could have been added to the matching, contradicting it's maximality.
If we denote by m the size (number of edges) of any matching and u the number of unmatched vertices, then the graph has $|V| = 2m + u$. If M and U represent these quantities for a maximum matching then we have $M \geqslant m$. These in-/equations imply that $m + u \geqslant M + U$. Here $M + U$ is the size of the edge cover we built from the maximum matching. If we recall that we can obtain a matching of any minimum edge cover by selecting an edge per star, then if m is the size of this matching, $m + u$ is the size of a minimum edge cover and we've just found that $M + U$ is also minimum. So the edge cover we built from the maximum matching is a minimum edge cover.

2 Hall's theorem

Consider a collection of n sets S_i that may intersect. Our goal is to find **distinct representatives** of this set, in the sense that we find distinct $s_i \in S_i$ for each set S_i. The trouble seems to arise when we pick these representatives without care: one set may run out of elements not already chosen as representatives for other sets, if it intersects with many other sets.

We can represent the problem as a bipartite graph, where the n sets S_i are represented by bipartition set U, and the elements of the üniverse"$\cup_{i\in[n]} S_i$ are represented by bipartition set W. we include the edge $\{S_i, s\}$ if $s \in S_i$. Then finding a system of distinct representatives is equivalent to finding a matching of U into W, which is a set of edges with no common endpoints that covers all of U. *Under what assumptions is there such a matching ?*

Let's start by looking for some easy necessary conditions that such a matching must satisfy. Since we ïnject"U into W, we must expect $|U| \leqslant |W|$. More precisely, we have $|U| \leqslant |N(U)|$ for the neighbourhood of U. In fact, this is true for any subset of U, so that for all $S \subseteq U$, we have $|S| \leqslant |N(S)|$.
The amazing thing is that this condition is already sufficient:

Hall's theorem:

If $|S| \leqslant |N(S)|$ for all $S \subseteq U$, then we can match U into W.

We'll try induction on the size of U by matching its elements one at a time.
In the initial case of $|U| = 1$, the condition says that the single element of u has a neighbour, that we can then match it with to get the desired result.
For the step, we'd like to match a vertex u of U to some $w \in W$ so that the condition $|S| \leqslant |N(S)|$ is maintained. The only way that $|S| \leqslant |N(S)|$ can be violated by such a move is if we had $|S| = |N(S)|$ for some S for which $w \in N(S)$ but $u \notin S$, as after deletion of u and w, we'd have $|S| > |N(S)|$, as $|N(S)|$ decreased by 1, but $|S|$ didn't.

In fact, the case in which $|S| < |N(S)|$ for all (non-empty proper) $S \subseteq U$ can be handled by taking any $u \in U$ and matching it to one of its neighbours, then deleting the pair and using induction to handle the remaining graph. To see that Hall's assumption is inherited by the remaining graph, take a set S for the initial graph and disjoin these cases. If S contained u, then $N(S)$ contained w, hence both sides of $|S| < |N(S)|$ decrease by 1 and the condition is maintained. If S didn't contain u, it may happen that $N(S)$ contains w, but even when $|N(S)|$ decreases by 1, we still have $|S| \leqslant |N(S)| - 1$ as they where integers. If $N(S)$ didn't contain w, then none of the sides of $|S| < |N(S)|$ decreased, and we keep the condition.

Now, how do we handle the problematic case of $|S| = |N(S)|$ for some $S \subseteq U$?
An idea is to use the induction hypothesis to match S into $N(S)$ and then match the rest $U \backslash S$ by induction again. The first problem in realising this approach is that to apply induction, S has to be proper and non-empty so as the sizes of S and $U \backslash S$ to be less then hat of U. This is where the assumption on S (in parentheses) in the previous paragraph came from. If the empty set and U are the only to satisfy $|S| = |N(S)|$, then we are in that case that we already handled.
The next step in successfully applying the induction assumption is to check Hall's condition. In the graph induced by the S and $N(S)$, for which $|S| = |N(S)|$, we also have $|S'| \leqslant |N(S')|$ for all $S' \subseteq S$, as they are also subsets of U. So we can match S into $N(S)$.
Now, we ask if the graph induced by $U \backslash S$ and $W \backslash N(S)$ has this property too. If some $S' \subseteq U \backslash S$ satisfied $|S'| > |N(S') \cap W \backslash N(S)|$, then by adding $|S| = |N(S)|$ on both sides, using disjointness of S' and S, and the fact that $N(S \cup S')$ can be partitioned into $N(S)$ and $N(S') \cap W \backslash N(S)$, we'd get contradiction $|S \cup S'| > |N(S \cup S')|$ to the Hall condition on the initial graph.
Therefore, we may apply induction and all cases worked out.

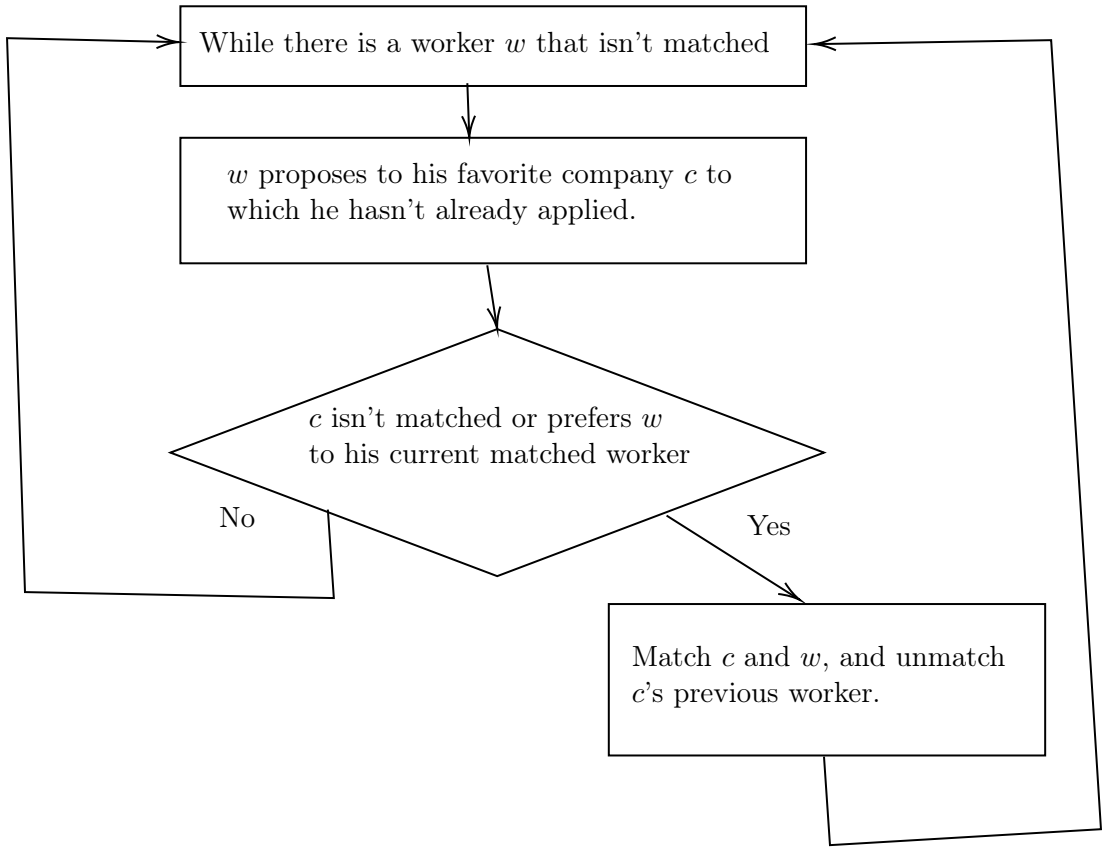
COMPLETE: add the other proofs from Diestel ?

3 The Gale-Shapely algorithm

In the stable matching problem, we're given n companies and n workers, which we hope to match 1-to-1. The twist is that both the companies and the workers have preferences on their choices: each company c has ranked the the workers w by a strict order, and vice-versa. How do we get a matching that accounts for these preferences ?
One idea is to avoid situations that are inefficient. The case we want to avoid is that of matching companies c and c' and workers w and w' respectively, when both c has preference $w' >_c w$ (it prefers w' to w) and w' has preference $c >_{w'} c'$. If this happens, our matching system would be circumvented by w' and c, leaving

us with more possibly dissatisfied matchings. A matching in which this doesn't occurs is called a **stable matching**.

The Gale-Shapely algorithm:
The Gale-Shapely algorithm simulates in a sense the real world. We let the workers apply for their favorite companies, and the companies stick to a worker as long as no prefered one applies:



We make two remarks: over the course of execution, the companies that a worker applies to gets worse in his ranking, and the workers a company is matched with gets better in their ranking.
From the workers perspective, this can be seen from the second box from the top of the flowchart, and for the companies, it's the disjunction box. This will ensure termination, as each worker can change company at most n times, increasing in preferences, and since there are n workers, we have at most n^2 steps. Next, note that all workers are matched at termination, due to the exit condition of the while loop, and since there are $n$companies as well, it's a perfect matching.

Is the final matching a stable one ? Assume for contradiction that there is matching of companies c and c' and workers w and w' respectively, when both c has preference $w' >_c w$ and w' has preference $c >_{w'} c'$. Since w' is matched with c' at termination, this company is the last one he applied to. When he applied, c already was matched to a preferred worker $w'' >_c w'$, since it rejected w' (flowchart disjunction). Since for companies, the workers get better, and company c ended up with w, we must have $w >_c w''$. By transitivity $w >_c w'$, which we assumed the opposite of.

4 Solutions

Ex.MaMu:
The symmetric difference $M_a \Delta M_u$ form even cycles or paths who's edges alternate between $M_a \backslash M_u$ and $M_u \backslash M_a$, for reasons explained in the proof of the characterisation of maximum matchings.
The paths can't be augmenting ones for M_u, as it's maximum, so each such path has at most one more edge in M_u then it does in M_a.
Next, each path must have at least one edge of M_a, otherwise the path is a single edge of M_u, which could be added to M_a without breaking the matching property, contradicting maximality. So there are at most $|M_a|$ paths.
Thus there are at most $1 \times |M_a|$ more edges in M_u then in M_a.
Since the maximum of edges the matchings can have in common is $|M_a|$, since $|M_a| < |M_u|$, we can conclude with $|M_u| = |M_u \cap M_a| + |M_u \backslash M_a| \leqslant 2|M_a|$. $\square$

Ex.BOC: First off, we can prove this for a connected graph, as bipartitions of the connected components of a graph can be merged to form a bipartition of all the graph.
So we may look at a spanning tree of the graph. The bipartition corresponds to the layers of the tree: we choose a root and partition the vertices on the parity of their distance to the root.
It could happen that when adding the edges not in the tree, vertices of the same partition become adjacent. But by the definition of our partitions, the cycle that this edge closes with the paths to the root in the tree must be odd, a possibility which we've excluded. So with all the edges considered, this bipartition remains valid. $\square$

Ex.EV: Suppose an exposed u has no augmenting path in M, but we can augment M with augmenting path P. If u had an augmenting path P' in the next iteration, meaning in matching $M \Delta P$, then P' has to meet P, otherwise it would have been an augmenting path for u in M already. Let's call v the first vertex common to P and P'. It can't be an endpoint of P, because this would imply that P' restricted to from u to v was an augmenting path in M, as v was still exposed in M. So v is in the interior of P. If we then consider the path that follows P' from u to v and then then follows the path P to it's endpoint in the direction for which the edges at v are either both in the matching $M \Delta P$, or none of them are, then in the initial matching M, this would be an augmenting path for u.
In all cases, we reach a contradiction, so u won't have an augmenting path in the next iteration. Inductively, we seen that u won't have an augmenting path in any future iteration, so we may ignore it in the future iterations of the algorithm. $\square$

Ex.MD: Dominoes cover two adjacent tiles, so if we model the problem by a graph who's vertices are the finite subset of $\mathbb{Z}^2$ and who's edges connect two vertices of from (n, m) which have one coordinate in common and the other coordinates at distance 1 (for example (2,3) and (3,3) are connected), then a cover by dominoes corresponds to a matching of this graph. So we're actually dealing with a matching problem.
To see that the underlying graph is bipartite, notice that diagonal tiles can't be covered by dominoes. So we partition vertices (of from (n, m)) on whether the parity of their coordinates coincides or not and see that all edges are in the cut of the partition, in the sense that neighboring vertices are in different partition sets. Think of the coloring of a chess board. So we're actually dealing with a bipartite matching problem.
$\square$

Ex.BlBl: All vertices on the blossom are incident to an edge of the matching which is on the blossom, except for the basis of the blossom. So the basis is the only vertex which can be incident to an edge of M not on the blossom: when we shrink the blossom, all other edges incident to the blossom can't be in M, so b is incident to at most one edge of M, which therefore maintains the matching property.
If an augmenting path of G/B starts or ends in b, this means that b is unmatched, so that the basis of the blossom is unmatched too. Then we can always prolong this path until it reaches the basis, circling the path in the direction that maintains alternation of edges until we arrive at the basis. This will produce an augmenting path.
If an augmenting path of G/B has b as an inner vertex, the path passes through the basis and some other point of the blossom. Again, we chose circle the blossom from the basis until we meet the other incidence vertex, circling in the direction such that the last edge on the blossom is one of M.
If an augmenting path of G/B doesn't pass through b or the entry and exit vertex on the blossom is the basis of the blossom, then no adjustments need to be made to the path for it to be augmenting in G.